

CSE 406 Project Report : YARA

1805044 - MD Misbahul Hasan Jinan

1805060 - Md Shamim Al Mamun

13 September 2023

1 Introduction

YARA is a very popular open-source and multi-platform tool (it works with most hosts running Windows, Linux, or Mac operating systems) that provides a mechanism to exploit code similarities between malware samples within a family.

It is a pattern-matching framework used to identify and classify malware and other IT security threats.

It can match certain patterns in selected directory or file and can say us if that directory contains a malware or not.

We can match patterns by either direct string comparison or hexadecimal to string comparison for more flexible pattern matching.

2 Yara Rules

To match patterns in files we need to write rules in Yara files(file extension : .yar).

An Yara rule contains 3 parts:

Meta: contains necessary information which doesn't have any effect in pattern matching.

Strings: This part contains all the list of patterns that we will need to match. These patterns can be either in string format or in Hexadecimal form. We can also use regular expression to represent certain pattern.

Condition: Here we set all the conditions using our patterns, as we wanna match them in the files.

3 Running and Compiling Yara Rules in Terminal

To compile and run yara files in command line we need to write this command:

```
yara < yarafilename > < FilenameThatWeWantToCheck >
```

For example : `yara myRules.yar mydownloadedfile.txt`

This command will search for patterns in mydownloadedfile.txt file according to the rules declared in the yara file myRules.txt.

Or if we want to search for malwares in a directory we need to write:

```
yara < yarafilename > < DirectoryNameThatWeWantToCheck >
```

Again if we want to see the matched patterns too, the command is:

```
yara - s < yarafilename > < File/DirectoryName >
```

In this report we will check for files with malicious patterns in our current directory. So the command will be :

```
yara < YaraFileName > ./
```

4 Pattern Matching Using String

If we wanna match patterns using string, we need to declare some string variables that we need to use in our conditions in the strings part of the rule. Then we write our necessary conditions that we need to match the patterns. For example:

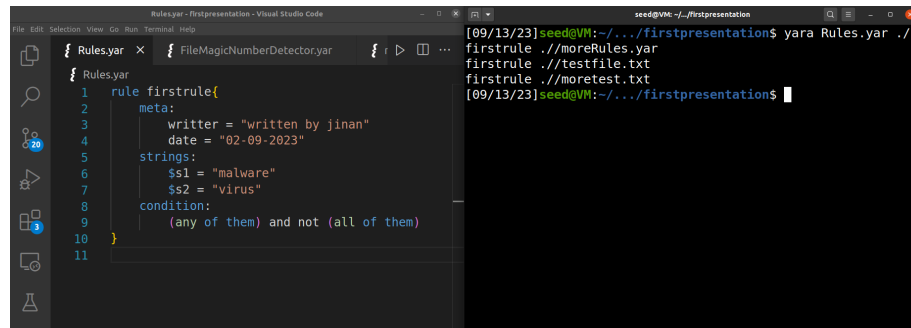


Figure 1: Basic YARA rule using string's pattern matching

This rule will match files who has at least one of the strings between "virus" and "malware" but not all of them.

If we run this in the command line we saw that it matched three files.

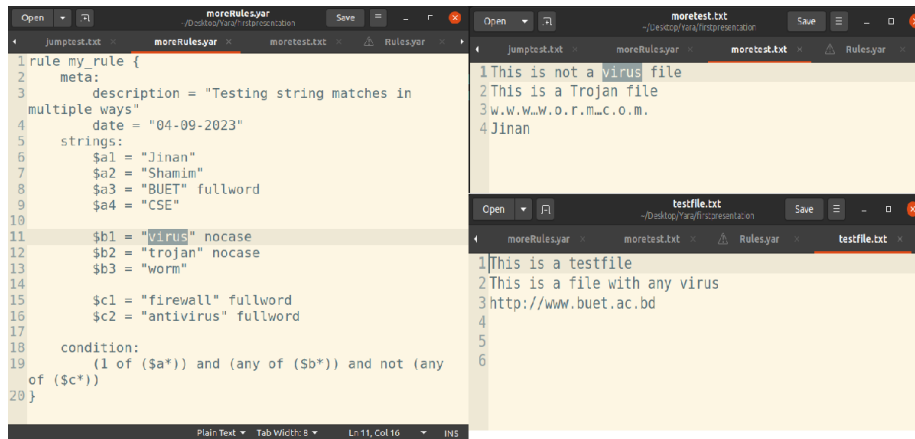


Figure 2: Files matched with the Rules.yar file's conditions

From the last figure we see that all the matched files contains one of the mentioned strings.

We can use more precise checking of string matching like in the next picture:

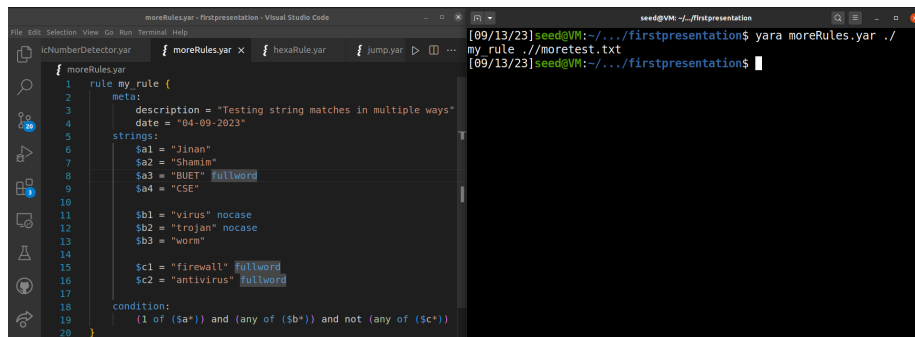


Figure 3: Precise string match checking

As we saw in the code This rule will match files with one of the strings from a1, a2, a3, a4 and any strings from b1, b2, b3 and with no strings from c1 and c2.

The matched files are:

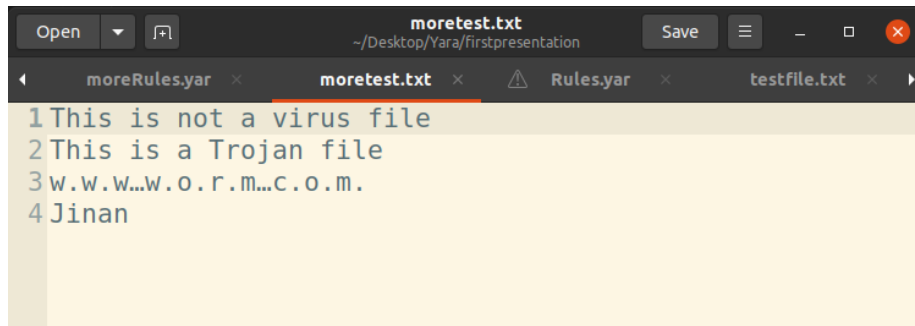


Figure 4: Matched files with moreRules.yar file's conditions

5 Patterns in Hexadecimal Form

We can use hexadecimal forms of ASCII characters to match any pattern. for that we will write one byte after another within the curly braces to represent hexadecimal string form. An example:

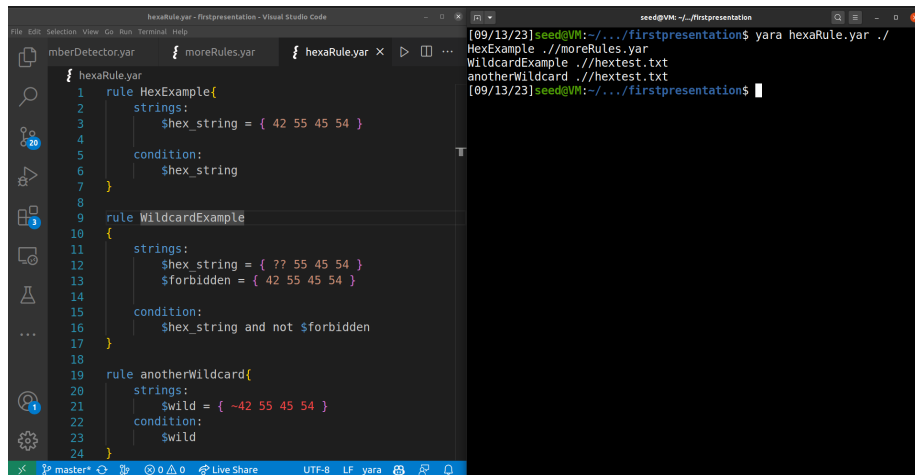


Figure 5: Writing patterns in hexadecimal form

The hex pattern 42 55 45 54 is equivalent to "BUET" in ASCII form. So the first rule HexExample will match files containing the string "BUET".

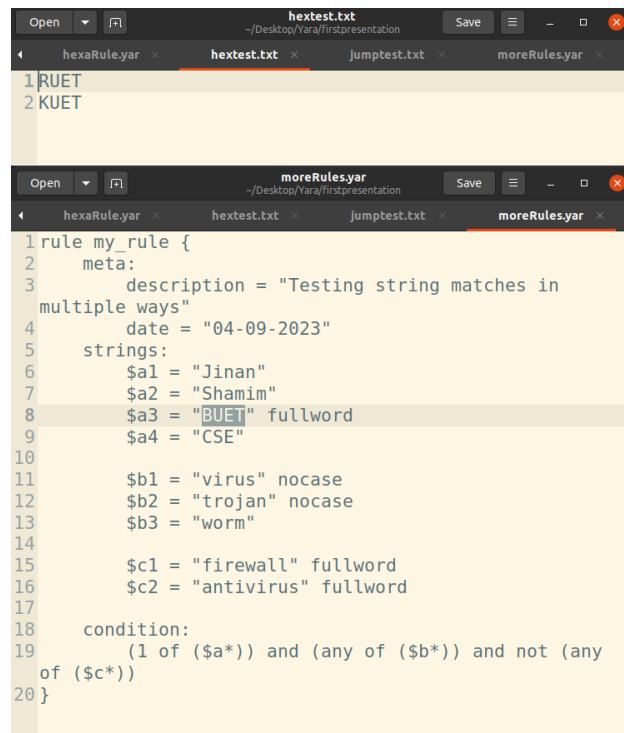
Wildcards:

In the next rule named WildcardExample we use another feature provided by yara called wildcard. Wildcard is a functionality to allow any character in a certain byte of the pattern by putting ?? in stead of a specific hex byte while

declaring a string. In our example rule we used our string and conditions in such a way that it will find for the String *UET (any character in *) but specifically not for BUET. We will see in our results that it matches such a file.

Powerful wildcards with not operator:

In our last rule named anotherwildcard we used not operator in the first byte of the string. 42 means not B. So, it will allow anything but B in the first place of the pattern. That means it will allow *UET only omitting BUET.



```
1 RUET
2 KUET

rule my_rule {
  meta:
    description = "Testing string matches in
multiple ways"
    date = "04-09-2023"
  strings:
    $a1 = "Jinan"
    $a2 = "Shamim"
    $a3 = "BUET" fullword
    $a4 = "CSE"

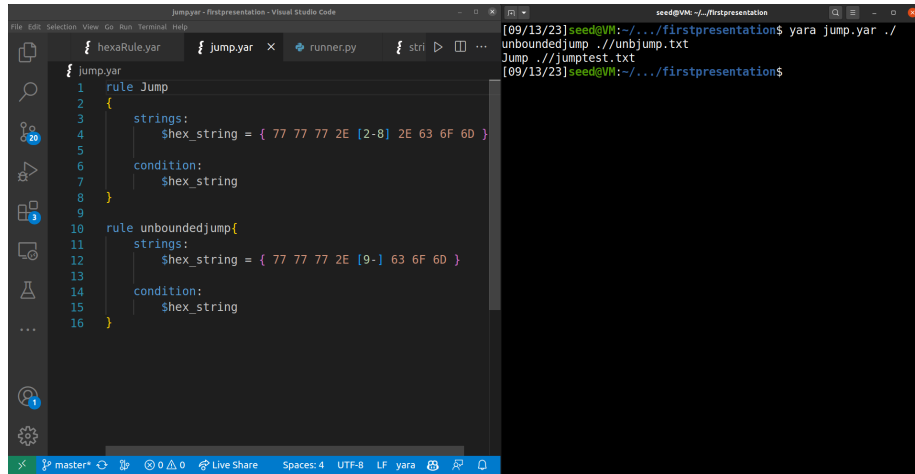
    $b1 = "virus" nocase
    $b2 = "trojan" nocase
    $b3 = "worm"

    $c1 = "firewall" fullword
    $c2 = "antivirus" fullword
  condition:
    (1 of ($a*)) and (any of ($b*)) and not (any
of ($c*))
}
```

Figure 6: Matched files with hexaRule.yar file's conditions

6 Using Jumps in Hexadecimal Form:

We can use jumps in hexadecimal form to say that there can be any substring of that length.



The screenshot shows a Visual Studio Code editor with two files open: `hexaRule.yar` and `jump.yar`. The `jump.yar` file contains two YARA rules. The first rule, `rule Jump`, has a `strings` block with `$hex_string = { 77 77 77 2E [2-8] 2E 63 6F 6D }` and a `condition` block with `$hex_string`. The second rule, `rule unboundedjump`, has a `strings` block with `$hex_string = { 77 77 77 2E [9-] 63 6F 6D }` and a `condition` block with `$hex_string`. To the right, a terminal window shows the command `yara jump.yar ./` being executed, with output showing the rules being processed.

```
1 rule Jump
2 {
3   strings:
4     $hex_string = { 77 77 77 2E [2-8] 2E 63 6F 6D }
5
6   condition:
7     $hex_string
8 }
9
10 rule unboundedjump{
11   strings:
12     $hex_string = { 77 77 77 2E [9-] 63 6F 6D }
13
14   condition:
15     $hex_string
16 }
```

```
[09/13/23]seed@VM:~/firstpresentation$ yara jump.yar ./
unboundedjump ./unjump.txt
Jump ./jumptest.txt
[09/13/23]seed@VM:~/firstpresentation$
```

Figure 7: Example of rules using jump functionality

For example: in the JumpExample rule
77 77 77 2E = “www.” and 2E 63 6F 6D = “.com”
So, this will match any pattern who has “www.(any 2 to 8 bytes long substring).com”

In the 2nd rule that’s an unbounded jump. It can be replaced by any substring of length 9-256 bytes.
So, this will match any pattern who has “www.(any 9 to 256 bytes long substring).com”

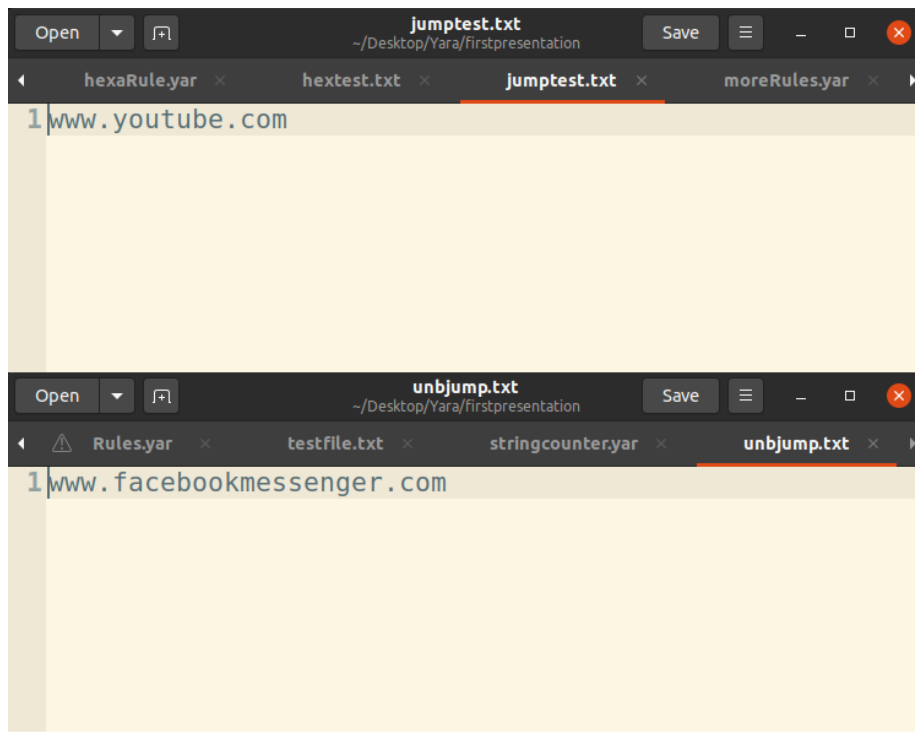


Figure 8: Matched files with jump.yar file's conditions

7 Counting Number of Particular String in a file

We can also match patterns by counting certain number of patterns in a file.

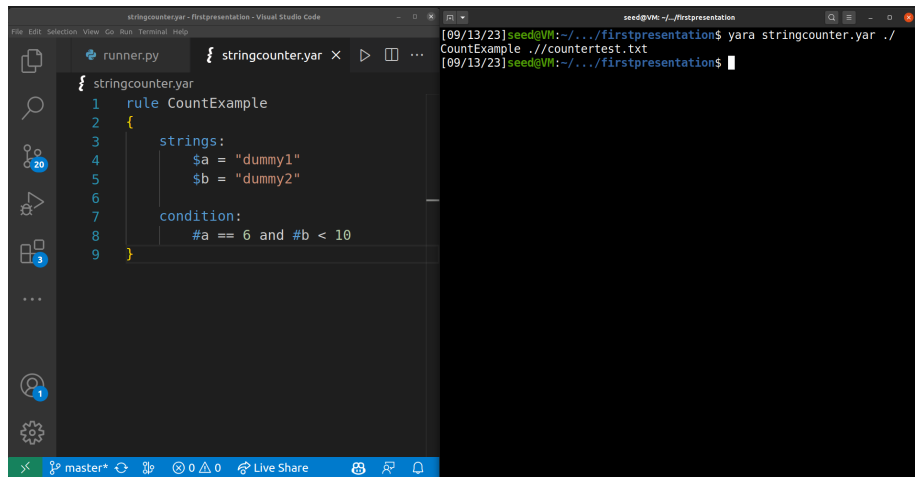


Figure 9: Example of a rule that can count number of a particular string in a file

In our example we can see that it matched the files with the word dummy1 6 times and dummy2 less than 10 times.

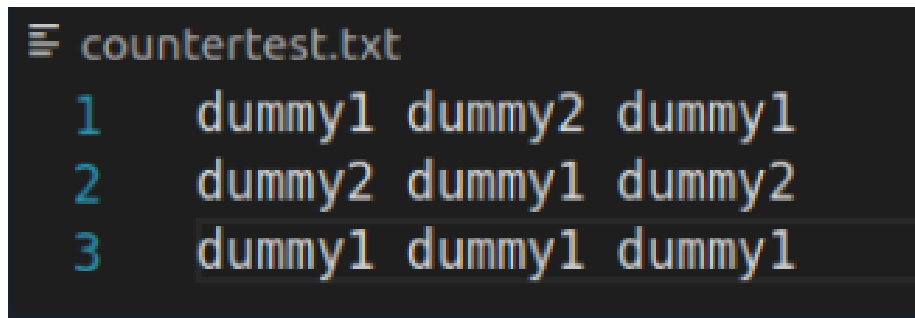
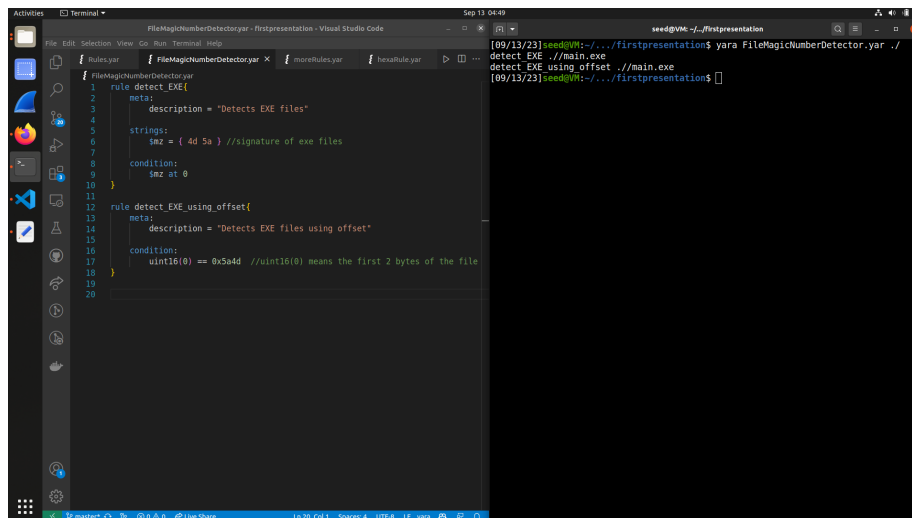


Figure 10: Matched file with the previous rule

8 Signature Based file Detection

Yara can detect files based on their signatures/ magic numbers.



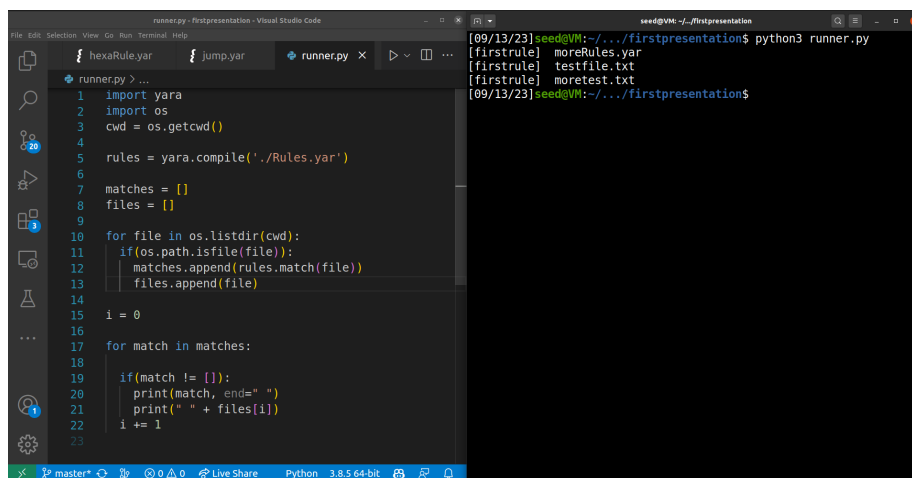
```
1 rule detect_EXE{
2   meta:
3     description = "Detects EXE files"
4
5   strings:
6     $mz = { 4d 5a } //signature of exe files
7
8   condition:
9     $mz at 0
10 }
11
12 rule detect_EXE_using_offset{
13   meta:
14     description = "Detects EXE files using offset"
15
16   condition:
17     uint16(0) == 0x5a4d //uint16(0) means the first 2 bytes of the file
18 }
19
20
```

```
[09/13/23]seed@VM:~/firstpresentation$ yara FileMagicNumberDetector.yar ./
detect_EXE ./main.exe
detect_EXE_using_offset ./main.exe
[09/13/23]seed@VM:~/firstpresentation$
```

Figure 11: Example of a rule that can detect file based on signature

For example all .exe files have a signature string “mz” in their first two characters. “mz” corresponds to 4d 5a in hexadecimal format. So, we can detect .exe, .bad, .dll files based on this feature.

9 Compiling and Running Yara Using Python



```
1 import yara
2 import os
3 cwd = os.getcwd()
4
5 rules = yara.compile('./Rules.yar')
6
7 matches = []
8 files = []
9
10 for file in os.listdir(cwd):
11     if(os.path.isfile(file)):
12         matches.append(rules.match(file))
13         files.append(file)
14
15 i = 0
16
17 for match in matches:
18
19     if(match != []):
20         print(match, end=" ")
21         print(" " + files[i])
22         i += 1
23
```

```
[09/13/23]seed@VM:~/firstpresentation$ python3 runner.py
[firstrule] moreRules.yar
[firstrule] testfile.txt
[firstrule] moretest.txt
[09/13/23]seed@VM:~/firstpresentation$
```

Figure 12: Example of compiling and running yara in python

We can compile and run yara files containing rules using python too. This gives us more flexibility while using yara rules. We can do much more using the features of python as we have seen in our example above.

We can use this feature to use output of yara code(the files) to use further in any code segment of python for more complex implementations.

10 Conclusion

This was some of our explored features of security tool YARA. After using this tool, we learned a way to find malware or suspicious files using pattern detection based algorithms implemented in yara.